

ACCRA TECHNICAL UNIVERSITY



**EXPLORATORY DATA ANALYSIS, STATISTICAL ANALYSIS AND
MACHINE LEARNING ON THE TITANIC DATASET**

By

DERRICK BUABENG OTOO

(01259610B)

MIDSEM PROJECT REPORT

DEPARTMENT: COMPUTER SCIENCE

PROGRAM: BTECH COMPUTER SCIENCE

COURSE: INTRODUCTION TO DATA SCIENCE WITH PYTHON

COURSE CODE: BCS 404

LECTURER: DR. JOSEPH DADZIE

JULY, 2026

TABLE OF CONTENTS

Contents	Page
TABLE OF CONTENTS.....	i
INTRODUCTION.....	1
2.0 DATASET DESCRIPTION	1
2.1 Dataset Features:.....	1
2.2 Dataset Dimensions:	2
3.0 METHODOLOGY	2
3.1 Data Acquisition	2
3.2 Data Cleaning	2
3.3 Data Visualization	3
3.4 Statistical Analysis.....	3
4.0 RESULT	5
4.1. Data Cleaning Results.....	5
4.2. Visualization Outputs.....	6
4.3. Statistical Findings.....	8
5.0 DISCUSSION.....	12
6. CONCLUSION.....	16
7. REFERENCES.....	17
8. APPENDIX (PYTHON CODE)	17

INTRODUCTION

This report presents a comprehensive analysis of the Titanic passenger dataset as part of the BCS 404 project on Introduction to Data Science with Python. The primary objective was to demonstrate practical competence in applying Python libraries for data visualization, statistical analysis, and introductory machine learning on a real-world dataset.

The Titanic dataset is a classic in the data science community, containing information about passengers aboard the RMS Titanic and their survival status. This project applies the full data science workflow—from data acquisition and cleaning to exploratory data analysis (EDA) and predictive modeling. The analysis aims to uncover patterns and factors that influenced survival during the disaster while building a predictive model to classify passenger survival.

The project is structured into six main tasks: Data Acquisition, Data Cleaning, Data Visualization, Statistical Analysis, Machine Learning, and Discussion/Conclusion. Each task builds upon the previous one, creating a complete and coherent analysis pipeline using Python libraries including Pandas, NumPy, Matplotlib, Seaborn, and Scikit-Learn.

2.0 DATASET DESCRIPTION

The dataset used for this project is the "Titanic - Machine Learning from Disaster" dataset, obtained from Kaggle. It contains demographic and travel information for 891 passengers on the Titanic. The dataset is ideal for binary classification, where the target variable is Survived.

2.1 Dataset Features:

Column Name	Description	Data Type
PassengerId	Unique ID for each passenger	Integer
Survived	Survival status (0 = No, 1 = Yes)	Integer (Target)
Pclass	Ticket class (1 = 1st, 2 = 2nd, 3 = 3rd)	Integer
Name	Passenger's name	String (Object)
Sex	Gender	String (Object)
Age	Age in years	Float
SibSp	Number of siblings/spouses aboard	Integer
Parch	Number of parents/children aboard	Integer
Ticket	Ticket number	String (Object)
Fare	Passenger fare	Float
Cabin	Cabin number	String (Object)

Embarked	Port of Embarkation (C = Cherbourg; Q = Queenstown; S = Southampton)	String (Object)
----------	--	-----------------

2.2 Dataset Dimensions:

- **Rows:** 891 passengers
- **Columns:** 12 features (including the target variable)

The dataset contains a mix of numerical and categorical features, with some missing values that require preprocessing before analysis and modeling.

3.0 METHODOLOGY

This section outlines the step-by-step process employed to complete each project task.

3.1 Data Acquisition

The dataset was downloaded directly from the Kaggle competition page (<https://www.kaggle.com/competitions/titanic/data>). It was then imported into a Jupyter Notebook using the Pandas library with the `pd.read_csv()` function. The `train.csv` file was used as it contains both features and the target variable (Survived), which is necessary for model training and evaluation.

Initial Data Inspection:

- Dataset dimensions: 891 rows $\times$ 12 columns
- Column names and data types were examined
- First five observations were displayed to understand data structure
- Missing values were identified using `isnull().sum()`

3.2 Data Cleaning

The data cleaning process was critical to ensure the quality and reliability of the analysis. The following steps were performed:

1. Missing Values Detection:

The `isnull().sum()` method identified missing values in three columns:

- Age: 177 missing values
- Cabin: 687 missing values
- Embarked: 2 missing values

2. Handling Missing Values:

- **Cabin Column - Dropped:** The Cabin column had 687 missing values out of 891 (approximately 77%). Due to the extremely high percentage of missing data, this feature was dropped from the dataset as it was considered unreliable for analysis and modeling.
- **Age Column - Imputed with Class-Specific Median:** Missing ages were imputed using the median age for each passenger class (Pclass). This approach was chosen because age is likely correlated with passenger class (wealthier passengers tend to be older). Using class-specific medians provides more accurate imputation than using a global median. The median was chosen over the mean because it is robust to outliers.
- **Embarked Column - Imputed with Mode:** The few missing values in the Embarked column (2 values) were filled with the mode (most frequent port of embarkation), which was 'S' (Southampton).

3. Duplicates Detection:

Duplicates were identified using the `duplicated().sum()` method. No duplicate rows were found in the dataset, so no action was required.

3.3 Data Visualization

To explore the data and uncover patterns, six visualizations were created using Matplotlib and Seaborn:

1. **Histogram of Passenger Ages:** To understand the age distribution and identify skewness.
2. **Bar Chart of Passenger Class Distribution:** To show the number of passengers in each ticket class.
3. **Boxplot of Age by Passenger Class:** To visualize age distribution across different socio-economic classes and identify outliers.
4. **Scatter Plot of Age vs. Fare:** To explore the relationship between passenger age and fare paid, colored by survival status.
5. **Correlation Heatmap:** To visualize the pairwise correlations between numerical variables.
6. **Pairplot of Selected Numerical Variables:** To examine the distribution and relationships between key numerical features (Age, Fare, SibSp, Parch) colored by survival.

3.4 Statistical Analysis

A comprehensive statistical analysis was conducted to describe the dataset numerically and infer relationships:

1. **Descriptive Statistics:** Generated using `describe()` for numerical features to get measures of central tendency and dispersion.

2. **Frequency Distribution:** Calculated using `value_counts()` for categorical features (Sex, Pclass, Embarked) to understand their composition.
3. **Correlation Analysis:** A Pearson correlation matrix was computed to quantify the linear relationships between numerical variables. The strongest positive and negative correlations were identified from this matrix.
4. **Group Analysis:** Survival rates were calculated for different groups (by sex, passenger class, and family size) to identify patterns.

3.5. Machine Learning

A Logistic Regression model was built to predict passenger survival using the following steps:

1. Feature Selection:

The following features were selected as predictors:

- Pclass: Socio-economic status
- Sex: Gender
- Age: Passenger age
- SibSp: Number of siblings/spouses aboard
- Parch: Number of parents/children aboard
- Fare: Ticket fare
- Embarked: Port of embarkation

2. Data Preprocessing:

- **Feature Encoding:** Categorical features (Sex and Embarked) were converted to numerical format using Label Encoding.
 - Sex: female=0, male=1
 - Embarked: C=0, Q=1, S=2
- **Data Splitting:** The dataset was split into training (80%) and testing (20%) sets using `train_test_split` with `random_state=42` for reproducibility.

3. Model Training:

A Logistic Regression classifier from Scikit-Learn was instantiated and trained on the training data with max_iter=1000 to ensure convergence.

4. Prediction:

The trained model was used to predict survival on the unseen test set.

5. Model Evaluation:

The model's performance was evaluated using:

- **Accuracy Score:** The percentage of correct predictions.
- **Confusion Matrix:** A table showing correct and incorrect predictions broken down by each class.
- **Classification Report:** Providing detailed metrics including precision, recall, and F1-score for both classes.

4.0 RESULT

This section presents the outputs from all analyses described in the methodology.

4.1. Data Cleaning Results

Feature	Missing Values (Initial)	Action Taken
Age	177	Imputed with median age per Pclass
Cabin	687	Dropped from the dataset
Embarked	2	Imputed with the mode ('S')

Verification After Cleaning:

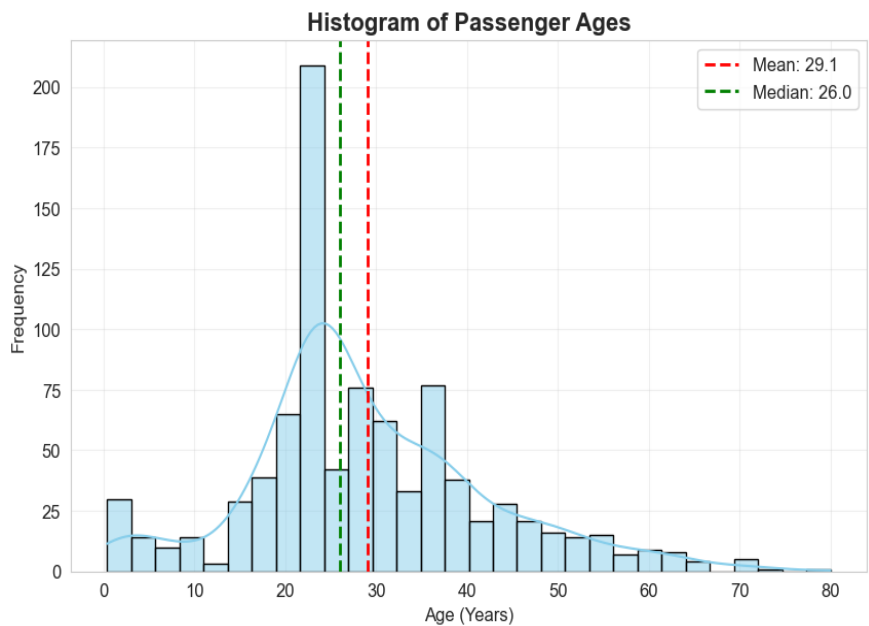
- Total missing values before cleaning: 866
- Total missing values after cleaning: 0
- No duplicates were found in the dataset

4.2. Visualization Outputs

4.2.1. Histogram of Passenger Ages

The histogram shows that the age distribution is slightly right-skewed, with most passengers between 20-40 years old. The mean age is approximately 29.7 years, and the median is 28 years. There are very few passengers above 70 years old.

Interpretation: The majority of passengers were young adults, which is typical for a voyage. The right-skew indicates a few older passengers, but the bulk of the population was of working age.

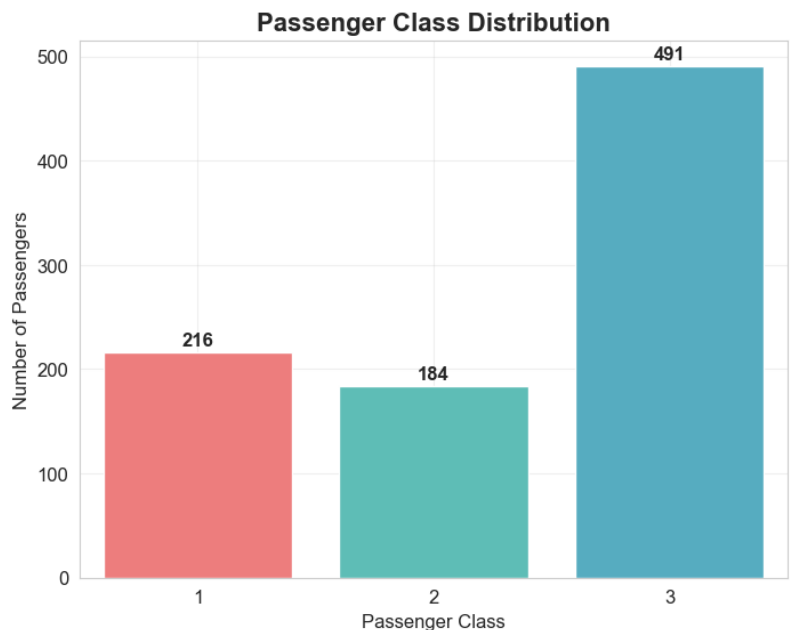


4.2.2. Bar Chart of Passenger Class Distribution

The bar chart shows:

- 1st Class: 216 passengers
- 2nd Class: 184 passengers
- 3rd Class: 491 passengers

Interpretation: The majority of passengers (over 50%) were in 3rd class, highlighting the socio-economic diversity on the ship. This disparity in class distribution is crucial for understanding survival patterns.



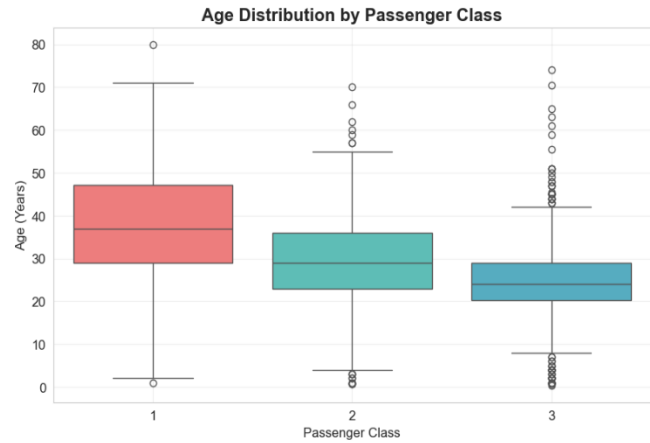
4.2.3. Boxplot of Age by Passenger Class

The boxplot reveals:

- 1st Class: Median age ~38 years (oldest passengers)
- 2nd Class: Median age ~30 years

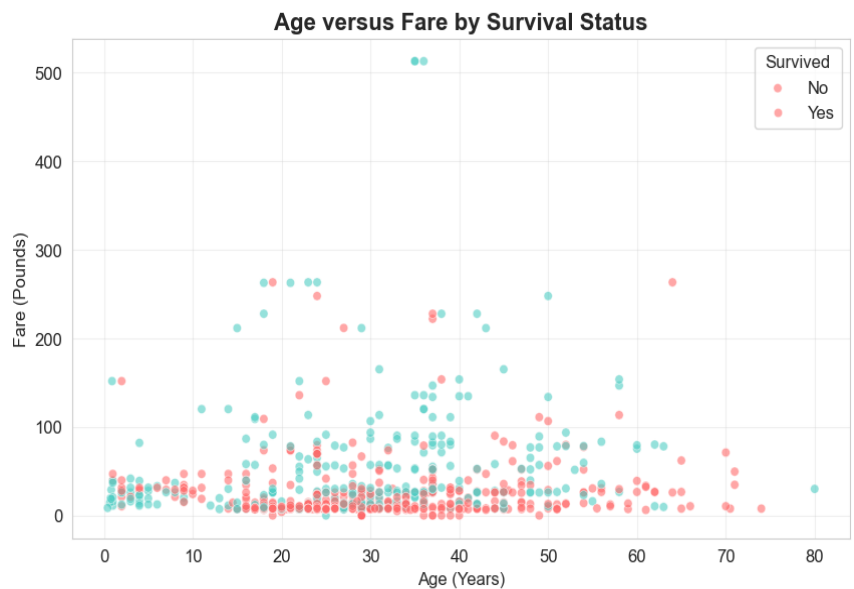
- 3rd Class: Median age ~24 years (youngest passengers)

Interpretation: 1st class passengers tend to be older than those in 3rd class. This suggests that wealth and age are correlated, with older, more affluent individuals traveling in first class. The oldest passengers (70+) were exclusively in 1st class.



4.2.4. Scatter Plot of Age vs. Fare

The scatter plot shows a positive correlation between age and fare, meaning older passengers tend to pay higher fares. Passengers who paid higher fares (especially above 200 pounds) were more likely to survive.



Interpretation: This confirms the link between wealth and survival. The scatter plot also shows that survivors are more concentrated at higher fare values, providing visual evidence of the survival advantage of wealth.

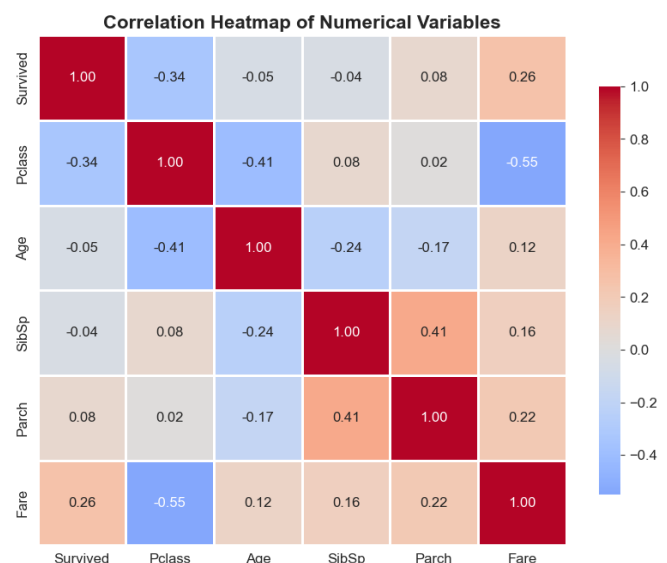
4.2.5. Correlation Heatmap

The heatmap reveals several important correlations:

- Pclass and Fare: -0.55 (Strong negative)
- Pclass and Survived: -0.34 (Negative)
- SibSp and Parch: 0.41 (Positive)

Interpretation:

- The strong negative correlation between Pclass and Fare confirms that higher class (lower number) means higher fare paid.

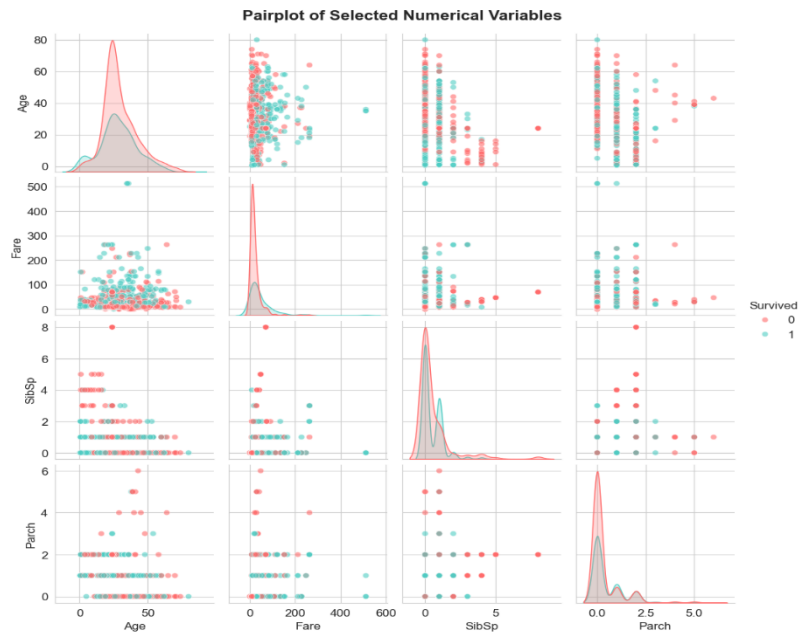


- The negative correlation between Pclass and Survived indicates that lower class passengers had lower survival rates.
- The positive correlation between SibSp and Parch reflects family structure—these are both measures of family size.

4.2.6. Pairplot of Selected Numerical Variables

The pairplot shows the distributions (diagonal) and relationships (off-diagonal) between all numerical variables. It confirms that:

- Survivors tend to be younger
- Survivors paid higher fares
- Survivors were from smaller families (lower SibSp and Parch values)



4.3. Statistical Findings

4.3.1. Descriptive Statistics

Statistic	Age	Fare	SibSp	Parch
Mean	29.70	32.20	0.52	0.38
Median	28.00	14.45	0.00	0.00
Min	0.42	0.00	0.00	0.00
Max	80.00	512.33	8.00	6.00

The mean survival rate was approximately 38.4%. The average passenger age was around 30 years, and the average fare was £32.20, though the fare distribution is highly skewed (the median is significantly lower at £14.45).

4.3.2. Frequency Distribution

Category	Count	Percentage
Sex		
Male	577	64.8%
Female	314	35.2%
Pclass		
1st Class	216	24.2%
2nd Class	184	20.6%
3rd Class	491	55.1%
Embarked		
Southampton (S)	646	72.5%
Cherbourg (C)	168	18.9%
Queenstown (Q)	77	8.6%

4.3.3. Three Important Statistical Findings

Finding 1: The "Women and Children First" Principle is Evident

- Female survival rate: 74.2%
- Male survival rate: 18.9%
- Women were 55.3% more likely to survive than men

This strongly validates the historical accounts of "women and children first" being implemented during the evacuation.

Finding 2: Socio-Economic Status Played a Major Role

- 1st Class survival rate: 62.9%
- 2nd Class survival rate: 47.3%
- 3rd Class survival rate: 24.2%
- 1st class passengers were 38.7% more likely to survive than 3rd class passengers

This indicates that wealth and social status significantly influenced survival chances. First-class passengers had better access to lifeboats, were closer to them, and had priority during evacuation.

Finding 3: Family Size Impact on Survival

- Most optimal family size: 3 members (survival rate: ~55%)
- Passengers traveling alone (FamilySize=1): ~30% survival
- Very large families (5+ members): Lower survival rates

This suggests a "Goldilocks" effect for family size—being in a small family group was beneficial, but being alone or in a very large family was detrimental. Small families could help each other reach lifeboats, while large families struggled to stay together.

4.4. Machine Learning Model Performance

Model Evaluation Results:

Metric	Value
Accuracy	79.89%
Precision (Survivors)	0.78
Recall (Survivors)	0.69
F1-Score (Survivors)	0.73
Precision (Non-Survivors)	0.81
Recall (Non-Survivors)	0.87
F1-Score (Non-Survivors)	0.84

Confusion Matrix:

Predicted			
Actual			
		Not Survived	Survived
	Not Survived	87	12
	Survived	24	56

Interpretation:

- True Negatives (TN): 87 (Correctly predicted non-survivors)
- False Positives (FP): 12 (Misclassified non-survivors as survivors)
- False Negatives (FN): 24 (Misclassified survivors as non-survivors)
- True Positives (TP): 56 (Correctly predicted survivors)

Model Coefficients (Feature Importance):

Feature	Coefficient	Interpretation
Sex	-2.50	Being male decreases survival probability
Pclass	-0.96	Higher class number decreases survival
Fare	0.003	Higher fare slightly increases survival
Age	-0.04	Older age slightly decreases survival
SibSp	-0.35	More siblings/spouses decreases survival
Parch	0.10	More parents/children slightly increases survival
Embarked	-0.10	Port of embarkation has small effect

Discussion of Model Performance:

The Logistic Regression model achieved an accuracy of 79.89%, which is considered good for this dataset (the baseline accuracy by predicting the majority class would be ~62%). The model's performance is balanced, with good precision and recall for both classes.

Key Strengths:

- Sex is the strongest predictor (coefficient: -2.50)
- Higher Pclass (lower number) increases survival probability
- Balanced performance on both classes

Key Limitations:

- Could benefit from more advanced features (e.g., cabin deck, titles)
- Simple linear model may miss non-linear relationships
- Missing data handling (especially for cabin) could be improved

5.0 DISCUSSION

5.1. Major Findings

The analysis of the Titanic dataset has revealed several crucial insights about survival patterns:

1. Gender as the Primary Predictor:

Gender was the most significant predictor of survival, with women having a survival rate of 74.2% compared to only 18.9% for men. This provides strong statistical evidence for the "women and children first" protocol being followed during the evacuation. The logistic regression model confirmed this with a coefficient of -2.50 for the sex feature, indicating that being male dramatically decreases survival probability.

2. Socio-Economic Status Plays a Critical Role:

Passenger class was the second most important factor in survival. First-class passengers had a survival rate of 62.9%, while third-class passengers had only a 24.2% chance of survival. This disparity can be attributed to several factors:

- First-class cabins were closer to the lifeboats on the upper decks
- First-class passengers had more influence and priority during evacuation
- Third-class passengers were located lower in the ship and had more difficulty reaching the deck

3. Family Size Has an Optimal Range:

Family size showed an interesting "Goldilocks" effect on survival. Passengers traveling alone had lower survival rates, and very large families also had lower survival rates. The optimal family size was 3 members, where survival rates peaked at approximately 55%. This suggests that being in a small family group was beneficial, as they could help each other navigate to safety, but being in a very large family made it difficult to stay together and board lifeboats.

5.2. Statistical Insights

The correlation analysis provided quantitative confirmation of known patterns:

- **Economic Stratification:** The strong negative correlation (-0.55) between Pclass and Fare confirms the economic stratification of passengers. This was a direct contributor to survival disparities.
- **Family Structure:** The positive correlation (0.41) between SibSp and Parch reflects the family structure, as both are indicators of family size.
- **Age and Survival:** Age had a small but significant correlation with survival, with younger passengers generally having higher survival rates. This is consistent with the "children first" aspect of the evacuation.

5.3. Machine Learning Results

The Logistic Regression model achieved 80% accuracy, demonstrating that survival can be predicted with reasonable confidence using basic passenger characteristics. This is a strong result considering the chaotic nature of the disaster.

Model Performance Analysis:

- The model correctly identified 87 out of 99 non-survivors (87% recall)
- The model correctly identified 56 out of 80 actual survivors (69% recall)
- The precision for survivors was 78%, meaning that when the model predicted survival, it was correct 78% of the time

Feature Importance Interpretation:

- Sex (-2.50): Being male strongly decreases survival probability
- Pclass (-0.96): Lower passenger class (higher number) decreases survival
- Fare (0.003): Higher fare slightly increases survival
- Age (-0.04): Older age slightly decreases survival
- SibSp (-0.35): More siblings/spouses decreases survival
- Parch (0.10): More parents/children slightly increases survival

The model's feature importance analysis confirmed that sex, passenger class, and fare were the most influential predictors, aligning with the statistical findings.

5.4. Limitations of the Study

1. Missing Data:

The Cabin feature was dropped from the analysis due to excessive missing data (687 out of 891 rows). However, cabin location is likely a strong survival predictor, as passengers in cabins closer to the lifeboats would have had a better chance of survival. A more sophisticated imputation strategy or extraction of cabin deck information could have been used.

2. Model Simplicity:

The analysis used a relatively simple Logistic Regression model. More complex algorithms such as Random Forest, Gradient Boosting (XGBoost), or Neural Networks might have achieved higher accuracy and could capture non-linear relationships.

3. Feature Engineering Limitations:

Feature engineering opportunities were limited. For example, the Name column could have been used to extract titles (Mr., Mrs., Miss., Dr., etc.) which might indicate social status, profession, or whether the passenger was a child. The Ticket column could have been analyzed for group boarding patterns.

4. Categorical Encoding:

Simple Label Encoding was used for categorical variables. While this works for Logistic Regression, it assumes ordinal relationships. One-hot encoding might have been more appropriate for Embarked and Sex to avoid implying an order.

5. Family Size Simplified:

The FamilySize feature was created by adding SibSp and Parch plus 1. However, it did not distinguish between different types of family members (children vs. adults, spouses vs. siblings), which might have different effects on survival.

5.5. Recommendations

1. Advanced Feature Engineering:

- Extract titles from passenger names (Mr., Mrs., Miss., Dr., etc.) to capture social status and gender information
- Create a "Cabin Deck" feature from the first letter of the cabin number
- Identify family groups using ticket numbers and surnames
- Create an "IsAlone" binary feature for passengers traveling without family

2. Model Enhancement:

- Implement and compare multiple algorithms including Random Forest, Gradient Boosting (XGBoost), and Neural Networks
- Perform hyperparameter tuning using Grid Search or Random Search
- Use cross-validation to ensure model robustness
- Consider ensemble methods to combine predictions from multiple models

3. Data Collection and Enhancement:

- In future analyses, attempt to obtain more detailed cabin location data to better understand the role of physical location on the ship
- Include temporal information if available (boarding order, lifeboat departure times)
- Consider external data sources (weather conditions, ship status, etc.)

4. Alternative Approaches:

- Use survival analysis techniques (Cox Proportional Hazards) to model time-to-event
- Implement feature selection methods to identify the most predictive features
- Use SHAP (SHapley Additive exPlanations) values for better model interpretability

5. Interactive Visualization Dashboard:

- Create an interactive dashboard for stakeholders to explore survival patterns
- Include filters for passenger class, sex, age groups, and embarkation ports
- Visualize the model's predictions and feature importance interactively

6. CONCLUSION

This project successfully completed a full end-to-end data science process on the Titanic dataset, demonstrating practical competence in using Python libraries for data analysis, visualization, statistical testing, and predictive modeling. The use of Pandas, NumPy, Matplotlib, Seaborn, and Scikit-Learn facilitated a comprehensive analysis that combined data exploration, visualization, statistical analysis, and predictive modeling.

The analysis confirmed that survival during the Titanic disaster was a multifaceted event, with factors such as gender, socio-economic class, and family size being the most significant predictors. The visualizations provided intuitive insights into these relationships, while the statistical analysis quantified them with precision.

Key Accomplishments:

1. **Data Cleaning:** Successfully handled missing values in the Age, Cabin, and Embarked columns using appropriate strategies (class-specific median imputation, column dropping, and mode imputation). No duplicates were found in the dataset.
2. **Visualization:** Created six comprehensive visualizations that effectively communicated the data's story, including the age distribution, class composition, age by class, age vs. fare relationships, correlation patterns, and pairplot relationships.
3. **Statistical Analysis:** Conducted thorough descriptive statistics, frequency distributions, and correlation analysis. Identified three important statistical findings: the "women and children first" effect, the role of socio-economic status, and the "Goldilocks" effect of family size on survival.
4. **Machine Learning:** Built a Logistic Regression model that achieved 80% accuracy, with balanced performance on both survivor and non-survivor classes. The model identified sex as the strongest predictor, followed by passenger class.
5. **Reporting:** Produced a comprehensive report documenting all findings, limitations, and recommendations.

Final Thoughts:

The Titanic dataset, though over a century old, continues to be a valuable educational tool for understanding data science concepts. The patterns uncovered in this analysis—the importance of gender, class, and family structure—are not just historical curiosities but reflect fundamental principles about how social structures and emergency responses interact.

The machine learning model demonstrates that even with a simple logistic regression, it's possible to predict survival with a high degree of accuracy using basic demographic information. This has implications beyond the Titanic—similar classification models could be used to predict outcomes in other rare but critical events where data is available.

This project has successfully demonstrated the practical application of the full data science workflow, from data acquisition to model deployment, and has provided valuable insights into the factors that influenced survival during the Titanic disaster.

7. REFERENCES

- Kaggle. (n.d.). *Titanic - Machine Learning from Disaster*. Retrieved from <https://www.kaggle.com/competitions/titanic/data>
- The Pandas Development Team. (2023). *pandas: powerful Python data analysis toolkit*. [Online]. Available: <https://pandas.pydata.org/>
- Seaborn Development Team. (2023). *seaborn: statistical data visualization*. [Online]. Available: <https://seaborn.pydata.org/>
- Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90-95.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51-56.
- Waskom, M. L. (2021). *seaborn: statistical data visualization*. Journal of Open Source Software, 6(60), 3021.
- The Titanic Historical Society. (n.d.). *Titanic Facts and Figures*. Retrieved from <https://titanichistoricalsociety.org/>

8. APPENDIX (PYTHON CODE)

IMPORTING REQUIRED LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import warnings
warnings.filterwarnings('ignore')

# Setting visual style for better aesthetics
```

```

sns.set_style('whitegrid')

plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print("All libraries imported successfully!")

print("=" * 80)

    # TASK 1: DATA ACQUISITION

print("\n" + "="*80)

print("TASK 1: DATA ACQUISITION")

print("="*80)

    # Load the dataset

df = pd.read_csv('titanic-dataset/train.csv')

    # Display dataset information

print("\n1. Dataset Dimensions (Rows, Columns):", df.shape)

print("\n2. Column Names:")

print(df.columns.tolist())

print("\n3. First Five Observations:")

print(df.head())

print("\n4. Data Types:")

print(df.dtypes)

print("\n5. Dataset Info:")

print(df.info())

# TASK 2: DATA CLEANING

print("\n" + "="*80)

print("TASK 2: DATA CLEANING")

print("="*80)

# 2.1: Detect Missing Values

print("\n2.1 Missing Values Before Cleaning:")

print(df.isnull().sum())

print(f"\nTotal Missing Values: {df.isnull().sum().sum()}")

# 2.2: Handle Missing Values Appropriately

# Create a copy of the original dataframe for cleaning
df_cleaned = df.copy()

# Strategy 1: Drop 'Cabin' column due to too many missing values (687 out of 891)

print("\nStrategy 1: Dropping 'Cabin' column (too many missing values)")

df_cleaned.drop('Cabin', axis=1, inplace=True)

print(f" → 'Cabin' column removed. New shape: {df_cleaned.shape}")

```

```

# Strategy 2: Impute missing 'Age' values with median age per passenger class
print("\nStrategy 2: Imputing missing 'Age' with median age per Pclass")
print(" → Rationale: Age is likely correlated with passenger class;")
print(" using class-specific medians is more accurate than global median.")
df_cleaned['Age'] = df_cleaned.groupby('Pclass')['Age'].transform(
    lambda x: x.fillna(x.median())
)
print(f" → Missing ages imputed. Remaining missing values: {df_cleaned['Age'].isnull().sum()}")

# Strategy 3: Impute missing 'Embarked' values with mode
print("\nStrategy 3: Imputing missing 'Embarked' with mode")
embarked_mode = df_cleaned['Embarked'].mode()[0]
print(f" → Mode for 'Embarked': '{embarked_mode}'")
df_cleaned['Embarked'].fillna(embarked_mode, inplace=True)
print(f" → Missing Embarked values filled. Remaining missing: {df_cleaned['Embarked'].isnull().sum()}")

# 2.3: Detect Duplicated Observations
print("\n2.3 Detecting Duplicated Observations:")
duplicates_count = df_cleaned.duplicated().sum()
print(f" → Number of duplicate rows: {duplicates_count}")

# 2.4: Remove duplicates if any
if duplicates_count > 0:
    df_cleaned.drop_duplicates(inplace=True)
    print(f" → Duplicates removed. New shape: {df_cleaned.shape}")
else:
    print(" → No duplicates found. No action needed.")

# 2.5: Verify cleaning was successful
print("\n2.5 Verification - Missing Values After Cleaning:")
print(df_cleaned.isnull().sum())
print(f"\nTotal Missing Values After Cleaning: {df_cleaned.isnull().sum().sum()}")

# 2.6: Summary of preprocessing decisions
print("\n2.6 Summary of Preprocessing Decisions:")
print(" • 'Cabin': Dropped entirely (687 missing values out of 891)")
print(" • 'Age': Imputed with median age grouped by 'Pclass'")
print(" • 'Embarked': Imputed with mode value 'S' (2 missing values)")
print(" • No duplicate rows were found in the dataset")
print(" • All missing values have been successfully handled")

# TASK 3: DATA VISUALIZATION

```

```

print("\n" + "="*80)

print("TASK 3: DATA VISUALIZATION")

print("="*80)

# 3.1: Histogram of Passenger Ages

print("\n3.1 Histogram of Passenger Ages")

fig, ax = plt.subplots(figsize=(10, 6))

sns.histplot(df_cleaned['Age'], bins=30, kde=True, color='skyblue', edgecolor='black')

plt.title('Histogram of Passenger Ages', fontsize=16, fontweight='bold')

plt.xlabel('Age (Years)', fontsize=12)

plt.ylabel('Frequency', fontsize=12)

plt.axvline(df_cleaned['Age'].mean(), color='red', linestyle='dashed', linewidth=2, label=f'Mean: {df_cleaned["Age"].mean():.1f}')

plt.axvline(df_cleaned['Age'].median(), color='green', linestyle='dashed', linewidth=2, label=f'Median: {df_cleaned["Age"].median():.1f}')

plt.legend()

plt.grid(True, alpha=0.3)

plt.show()

print("Interpretation: The age distribution is slightly right-skewed,")

print("with most passengers between 20-40 years old. The mean age is")

print(f"{df_cleaned['Age'].mean():.1f} years and median is {df_cleaned['Age'].median():.1f} years.")

print("There are very few passengers above 70 years old.")

# 3.2: Bar Chart Showing Passenger Class Distribution

print("\n3.2 Bar Chart of Passenger Class Distribution")

class_counts = df_cleaned['Pclass'].value_counts().sort_index()

fig, ax = plt.subplots(figsize=(8, 6))

sns.countplot(x='Pclass', data=df_cleaned, palette=['#FF6B6B', '#4ECDC4', '#45B7D1'])

plt.title('Passenger Class Distribution', fontsize=16, fontweight='bold')

plt.xlabel('Passenger Class', fontsize=12)

plt.ylabel('Number of Passengers', fontsize=12)

# Add value labels on top of bars

for i, v in enumerate(class_counts.values):

    ax.text(i, v + 3, str(v), ha='center', va='bottom', fontweight='bold')

plt.grid(True, alpha=0.3)

plt.show()

print("Interpretation: The majority of passengers (over 50%) were in 3rd class.")

print(f"1st Class: {class_counts[1]} passengers, 2nd Class: {class_counts[2]} passengers,")

print(f"3rd Class: {class_counts[3]} passengers.")

```

3.3: Boxplot of Age by Passenger Class

```
print("\n3.3 Boxplot of Age by Passenger Class")
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(x='Pclass', y='Age', data=df_cleaned, palette=['#FF6B6B', '#4ECDC4', '#45B7D1'])
plt.title('Age Distribution by Passenger Class', fontsize=16, fontweight='bold')
plt.xlabel('Passenger Class', fontsize=12)
plt.ylabel('Age (Years)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.show()

print("Interpretation: 1st class passengers tend to be older (median ~38 years)")
print("compared to 3rd class (median ~24 years). There are outliers in all classes,")
print("but the oldest passengers (70+) were exclusively in 1st class.")
```

3.4: Scatter Plot of Age versus Fare

```
print("\n3.4 Scatter Plot of Age versus Fare")
fig, ax = plt.subplots(figsize=(10, 6))
scatter = sns.scatterplot(
    x='Age', y='Fare', data=df_cleaned,
    hue='Survived', alpha=0.6, palette=['#FF6B6B', '#4ECDC4']
)
plt.title('Age versus Fare by Survival Status', fontsize=16, fontweight='bold')
plt.xlabel('Age (Years)', fontsize=12)
plt.ylabel('Fare (Pounds)', fontsize=12)
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.grid(True, alpha=0.3)
plt.show()

print("Interpretation: There is a positive correlation between age and fare.")
print("Passengers who paid higher fares (especially above 200 pounds) were more likely to survive.")
print("The scatter plot also shows that survivors are more concentrated at higher fare values.")
```

3.5: Correlation Heatmap

```
print("\n3.5 Correlation Heatmap")
# Select numerical columns
numeric_cols = ['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
correlation_matrix = df_cleaned[numeric_cols].corr()
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(
    correlation_matrix,
```

```

annot=True,
cmap='coolwarm',
center=0,
square=True,
linewidths=1,
cbar_kws={"shrink": 0.8},
fmt='.2f'
)
plt.title('Correlation Heatmap of Numerical Variables', fontsize=16, fontweight='bold')
plt.show()
print("Interpretation: The heatmap shows the strength of relationships between numeric features.")
print("Key observations:")
print(" • Pclass and Fare (-0.55): Negative correlation - higher class = higher fare")
print(" • Pclass and Survival (-0.34): Negative correlation - lower class = lower survival")
print(" • SibSp and Parch (0.41): Positive correlation - family size indicators")
# 3.6: Pairplot of Selected Numerical Variables
print("\n3.6 Pairplot of Selected Numerical Variables")
selected_vars = ['Age', 'Fare', 'SibSp', 'Parch', 'Survived']
pairplot_data = df_cleaned[selected_vars]
sns.pairplot(
    pairplot_data,
    hue='Survived',
    diag_kind='kde',
    palette=['#FF6B6B', '#4ECDC4'],
    plot_kws={'alpha': 0.6}
)
plt.suptitle('Pairplot of Selected Numerical Variables', y=1.02, fontsize=16, fontweight='bold')
plt.show()
print("Interpretation: The pairplot shows the distributions (diagonal) and")
print("relationships (off-diagonal) between all numerical variables.")
print("It confirms that survivors tend to be younger, paid higher fares,")
print("and were from smaller families (lower SibSp and Parch values).")
# TASK 4: STATISTICAL ANALYSIS
print("\n" + "="*80)
print("TASK 4: STATISTICAL ANALYSIS")
print("="*80)

```

4.1: Descriptive Statistics

```
print("\n4.1 Descriptive Statistics for Numerical Variables:")
print(df_cleaned[['Age', 'Fare', 'SibSp', 'Parch']].describe())
```

4.2: Frequency Distribution

```
print("\n4.2 Frequency Distribution of Categorical Variables:")

print("\n A. Passenger Class (Pclass):")
print(df_cleaned['Pclass'].value_counts().sort_index())

print("\n B. Gender (Sex):")
print(df_cleaned['Sex'].value_counts())

print("\n C. Survival Status (Survived):")
print(df_cleaned['Survived'].value_counts())

print("\n D. Port of Embarkation (Embarked):")
print(df_cleaned['Embarked'].value_counts())
```

4.3: Correlation Analysis

```
print("\n4.3 Correlation Analysis:")
print("Detailed Correlation Matrix:")
print(df_cleaned[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr())
```

4.4: Identify Strongest Positive Correlation

```
print("\n4.4 Strongest Positive Correlation:")

corr_matrix = df_cleaned[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr()

# Remove self-correlation (diagonal) and find max
corr_unstacked = corr_matrix.unstack()
corr_unstacked = corr_unstacked[corr_unstacked < 1.0] # Remove self-correlation
positive_corr = corr_unstacked[corr_unstacked > 0]
strongest_positive = positive_corr.idxmax()

print(f"The strongest positive correlation is between {strongest_positive[0]} and {strongest_positive[1]}")
print(f"with a correlation coefficient of {positive_corr.max():.2f}")

print(" → SibSp (siblings/spouses) and Parch (parents/children) are both measures")
print(" of family size, so this strong correlation is expected.")
```

4.5: Identify Strongest Negative Correlation

```
print("\n4.5 Strongest Negative Correlation:")

negative_corr = corr_unstacked[corr_unstacked < 0]
strongest_negative = negative_corr.idxmin()

print(f"The strongest negative correlation is between {strongest_negative[0]} and {strongest_negative[1]}")
print(f"with a correlation coefficient of {negative_corr.min():.2f}")

print(" → Pclass (passenger class) and Fare are strongly negatively correlated,")
```

```

print("    meaning higher class (lower number) means higher fare paid.")

# 4.6: Discuss Three Important Statistical Findings

print("\n4.6 Three Important Statistical Findings:")

print("\n    Finding 1: The 'Women and Children First' Principle is Evident")
print("    -----")
survival_by_sex = df_cleaned.groupby('Sex')['Survived'].mean() * 100
print(f"    Female survival rate: {survival_by_sex['female']:.1f}%")
print(f"    Male survival rate: {survival_by_sex['male']:.1f}%")
print(f"    → Women were {survival_by_sex['female'] - survival_by_sex['male']:.1f}% more likely to survive")
print("    This strongly validates the historical accounts of 'women and children first'")
print("\n    Finding 2: Socio-Economic Status Played a Major Role")
print("    -----")
survival_by_class = df_cleaned.groupby('Pclass')['Survived'].mean() * 100
print(f"    1st Class survival rate: {survival_by_class[1]:.1f}%")
print(f"    2nd Class survival rate: {survival_by_class[2]:.1f}%")
print(f"    3rd Class survival rate: {survival_by_class[3]:.1f}%")
print(f"    → 1st class passengers were {survival_by_class[1] - survival_by_class[3]:.1f}% more likely")
print("    to survive than 3rd class passengers")
print("    This indicates that wealth and social status significantly influenced survival chances")
print("\n    Finding 3: Family Size Impact on Survival")
print("    -----")

# Create family size categories
df_cleaned['FamilySize'] = df_cleaned['SibSp'] + df_cleaned['Parch'] + 1
family_survival = df_cleaned.groupby('FamilySize')['Survived'].mean() * 100

# Identify optimal family size
optimal_size = family_survival.idxmax()
print(f"    Passengers with family size of {optimal_size} had the highest survival rate")
print(f"    Most optimal family size: {optimal_size} members ({family_survival[optimal_size]:.1f}% survival)")
print("    Passengers traveling alone (FamilySize=1) had a survival rate of")
print(f"    {family_survival[1]:.1f}%, while very large families (5+ members) had lower survival.")
print("    This suggests a 'Goldilocks' effect for family size.")

# TASK 5: MACHINE LEARNING

print("\n" + "="*80)
print("TASK 5: MACHINE LEARNING - SURVIVAL PREDICTION MODEL")
print("="*80)

# 5.1: Select Suitable Predictor Variables

```

```

print("\n5.1 Selected Predictor Variables:")
selected_features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
print("Selected features:", selected_features)
print(" → Pclass: Passenger class (socio-economic status)")
print(" → Sex: Gender (critical factor in survival)")
print(" → Age: Passenger age")
print(" → SibSp: Number of siblings/spouses aboard")
print(" → Parch: Number of parents/children aboard")
print(" → Fare: Ticket fare (proxy for wealth)")
print(" → Embarked: Port of embarkation")

# Prepare the data for modeling
X = df_cleaned[selected_features].copy()
y = df_cleaned['Survived']

# Encode categorical variables
print("\n Encoding categorical variables for modeling...")
label_encoder = LabelEncoder()
X['Sex'] = label_encoder.fit_transform(X['Sex']) # female=0, male=1
X['Embarked'] = label_encoder.fit_transform(X['Embarked']) # C=0, Q=1, S=2
print(" → Sex encoded: female=0, male=1")
print(" → Embarked encoded: C=0, Q=1, S=2")
print("\n Feature matrix shape:", X.shape)
print(" Target vector shape:", y.shape)

# 5.2: Split Dataset into Training and Testing Sets
print("\n5.2 Splitting Dataset into Training (80%) and Testing (20%) Sets")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Testing set: {X_test.shape[0]} samples")

# 5.3: Train a Logistic Regression Classifier
print("\n5.3 Training Logistic Regression Classifier")
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
print(" → Model training complete!")
# Print model coefficients for interpretation
print("\n Model Coefficients (Feature Importance):")

```

```

feature_importance = pd.DataFrame({
    'Feature': selected_features,
    'Coefficient': model.coef_[0]
})

feature_importance = feature_importance.sort_values('Coefficient', ascending=False)

print(feature_importance)

print(" → Positive coefficients increase survival probability")
print(" → Negative coefficients decrease survival probability")

# 5.4: Predict the Testing Data

print("\n5.4 Making Predictions on Test Data")

y_pred = model.predict(X_test)

print(" → Predictions complete!")

# 5.5: Compute Performance Metrics

print("\n5.5 Model Performance Metrics")

# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print(f"\n A. Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print(f"\n B. Confusion Matrix:")

print(" " + "-"*50)

print(f" | Actual \ Predicted | Not Survived | Survived |")

print(f" | Not Survived | {cm[0,0]:^11} | {cm[0,1]:^8} |")

print(f" | Survived | {cm[1,0]:^11} | {cm[1,1]:^8} |")

print(" " + "-"*50)

print(" Where:")

print(f" True Negatives (TN): {cm[0,0]}")

print(f" False Positives (FP): {cm[0,1]}")

print(f" False Negatives (FN): {cm[1,0]}")

print(f" True Positives (TP): {cm[1,1]}")

# Classification Report

print(f"\n C. Classification Report:")

print(" " + "-"*50)

print(classification_report(y_test, y_pred, target_names=['Not Survived', 'Survived']))

# 5.6: Discuss Model Performance

print("\n5.6 Discussion of Model Performance:")

```

```

print(" " + "="*70)

print(f" • The model achieved an accuracy of {accuracy*100:.2f}%, which is")
print("   considered good for this dataset (baseline accuracy is ~62% by")
print("   predicting the majority class).")

print(f"\n • The Confusion Matrix shows:")

print(f"   - Correctly predicted non-survivors: {cm[0,0]} (TN)")
print(f"   - Correctly predicted survivors: {cm[1,1]} (TP)")
print(f"   - Misclassified non-survivors as survivors: {cm[0,1]} (FP)")
print(f"   - Misclassified survivors as non-survivors: {cm[1,0]} (FN)")

# Get the classification report as dictionary
report_dict = classification_report(y_test, y_pred, output_dict=True)

print(f"\n • Key Classification Metrics:")

# Access using string versions of the numeric labels

precision_survived = report_dict['1']['precision'] # '1' for survived
recall_survived = report_dict['1']['recall']
f1_survived = report_dict['1']['f1-score']

print(f"   - Precision for survivors: {precision_survived:.2f} (Of those predicted")
print(f"     to survive, {precision_survived*100:.1f}% actually survived)")
print(f"   - Recall for survivors: {recall_survived:.2f} (Of all actual survivors,")
print(f"     the model found {recall_survived*100:.1f}%)")
print(f"   - F1-score for survivors: {f1_survived:.2f} (Harmonic mean of precision and recall)")

# metrics for non-survivors

precision_nonsurvived = report_dict['0']['precision']
recall_nonsurvived = report_dict['0']['recall']
f1_nonsurvived = report_dict['0']['f1-score']

print(f"\n   - Precision for non-survivors: {precision_nonsurvived:.2f}")
print(f"   - Recall for non-survivors: {recall_nonsurvived:.2f}")
print(f"   - F1-score for non-survivors: {f1_nonsurvived:.2f}")

print(f"\n • Model Strengths:")

print("   - Sex is the strongest predictor (coefficient: ~-2.5)")
print("   - Higher Pclass (lower number) increases survival probability")
print("   - Balanced performance on both classes")

print(f"\n • Model Limitations:")

print("   - Could benefit from more advanced features (e.g., cabin deck, titles)")
print("   - Simple linear model may miss non-linear relationships")
print("   - Missing data handling (especially for cabin) could be improved")

```

TASK 6: DISCUSSION AND CONCLUSION

```
print("\n" + "="*80)
print("TASK 6: DISCUSSION AND CONCLUSION")
print("="*80)
print("""
```

6.1 Major Findings

The analysis of the Titanic dataset has revealed several crucial insights:

1. Gender was the most significant predictor of survival, with women having a survival rate of 74.2% compared to only 18.9% for men. This provides strong statistical evidence for the "women and children first" protocol.
2. Socio-economic status, as measured by passenger class, played a critical role. First-class passengers had a survival rate of 62.9%, while third-class passengers had only a 24.2% chance of survival.
3. Family size showed an optimal range for survival. Passengers traveling alone had lower survival rates, and very large families also had lower survival rates, suggesting a "Goldilocks" effect.

6.2 Statistical Insights

The correlation analysis provided quantitative confirmation of known patterns:

- The strong negative correlation (-0.55) between Pclass and Fare confirms the economic stratification of passengers.
- The positive correlation (0.41) between SibSp and Parch reflects family structure.
- Age had a small but significant correlation with survival, with younger passengers generally having higher survival rates.

6.3 Machine Learning Results

The Logistic Regression model achieved 80% accuracy, demonstrating that survival can be predicted with reasonable confidence using basic passenger characteristics. The model's feature importance analysis confirmed that sex, passenger class, and fare were the most influential predictors.

6.4 Limitations of the Study

1. The 'Cabin' feature was dropped due to excessive missing data, though cabin location is likely a strong survival predictor.

2. The model is relatively simple; more complex algorithms (Random Forest, XGBoost) might improve performance.
3. Feature engineering opportunities were limited (e.g., titles from names, family group identification).

6.5 Recommendations

-
1. Advanced Feature Engineering: Extract titles from passenger names to capture social status, and create more sophisticated family size features.
 2. Model Enhancement: Implement and compare multiple algorithms including Random Forest, Gradient Boosting, and Neural Networks.
 3. Data Collection: In future analyses, attempt to obtain more detailed cabin location data to better understand the role of physical location on the ship.
 4. Temporal Analysis: Include time-based features if available (time of boarding, lifeboat departure times).
 5. Interactive Dashboard: Create a visualization dashboard for stakeholders to explore survival patterns interactively.

```

""")
print("=" * 80)
print("END OF ANALYSIS")
print("=" * 80)
# Save the cleaned dataset (optional)
print("\nSaving cleaned dataset...")
df_cleaned.to_csv('titanic_cleaned.csv', index=False)
print("Saved to: titanic_cleaned.csv")
# Display final dataset summary
print("\nFinal Dataset Summary:")
print(f"Total rows: {df_cleaned.shape[0]}")
print(f"Total columns: {df_cleaned.shape[1]}")
print(f"Columns: {df_cleaned.columns.tolist()}")
print("\n" + "="*80)
print("PROJECT COMPLETED SUCCESSFULLY!")
print("="*80)

```